Phase 3 | Technologies and Design Decisions

Explain each technology, the boundary it owns, and the trade-offs in the current design.

React/Vite Edition | Repository-grounded study material | Snapshot: 2026-10-24

1. Main technology choices

Technology	Purpose	Reasoning and trade-off
React	Component-based UI and shared state.	Reusable dashboard and learning components; requires clear state ownership.
Vite	Dev server and production bundler.	Fast local feedback; emits static assets that need SPA rewrites or Express hosting.
React Router	Client-side navigation.	Keeps route transitions in the browser; direct refresh depends on host fallback config.
Tailwind CSS	Utility-based UI styling.	Rapid iteration; shared design tokens would improve consistency.
Node.js and Express	REST API, middleware, auth and integration endpoints.	Simple service boundary; input validation and consistent errors need continued attention.
MongoDB Atlas and Mongoose	Cloud document storage and object mapping.	Flexible user records; indexes, schema discipline and access controls remain important.
Passport and Google OAuth	Federated sign-in.	Delegates identity verification; callback URLs, state and account linking must be secure.
Flask and Gemini	Python chatbot integration.	Isolates model calls; adds another service, URL, secret and availability dependency.
Leaflet and Geoapify	Map rendering and nearby POI search.	Rendering and data retrieval are separate concerns; provider key/quota can fail.
Phaser 3	Earthquake drill simulation.	Provides game scenes, inputs, collisions and timing for interactive practice.

2. Design boundaries

- React/Vite builds the user experience; Express handles the Node API and can serve static assets.
- MongoDB Atlas stores application records; Mongoose defines schema and persistence operations.
- Passport performs Google OAuth; the application still issues its own JWT after identity verification.
- Leaflet renders tiles, controls and markers. Geoapify Places performs POI lookup. OpenStreetMap supplies map data and attribution context.
- Flask wraps the Gemini provider call so its key is not bundled into the Vite frontend.

3. Data contracts and normalization

Third-party providers differ in field names and formats. The backend alert route creates normalized summary objects and parses CAP XML to JSON. The nearby route maps Geoapify GeoJSON features into a compact coordinate/category shape. The UI then handles filtering, display labels, distance calculation and map markers. Stable internal contracts reduce coupling to provider response formats.

4. Why not use a single backend?

Keeping the chatbot in Flask can be useful when the team already has Python code or wants to isolate model integration. Its cost is extra deployment and operational complexity. Consolidating the chatbot into Express would reduce runtime count but require porting provider logic. The right choice depends on team skills, observability, deployment cost and expected traffic.

5. Design improvements to discuss

- Define request/response schemas and use consistent error envelopes.
- Add startup validation for required environment variables without printing secret values.
- Use shared cache infrastructure only when multi-instance consistency is needed.
- Make alert freshness explicit and separate an empty feed from a failed feed.
- Keep secret values server-side, rotate exposed credentials, and use deployment secret settings.
- Document external provider attribution, quotas and fallback behavior.